



HOW TO INSTALL K-BUS

DRIVER IN A BECKHOFF

CX1020 (RTAI)

v.0.1

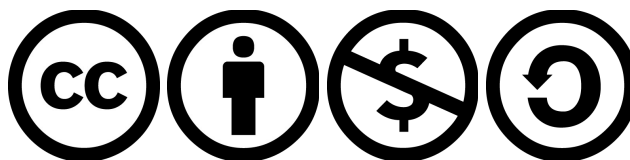
Authors: J. Andrés Cubillos Galvis

Ekaitz Larzabal Balerdi

Mikel Larrea Sukia

Eloy Irigoyen Gordo

Date:09/05/12



This work is licensed under the Creative Commons Atribución-NoComercial-CompartirIgual 3.0 Unported License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-sa/3.0/> or send a letter to Creative Commons, 444 Castro Street, Suite 900, Mountain View, California, 94041, USA.

Contents

GENERAL INFORMATION.....	4
PRELIMINARIES.....	5
STEP 1: INSTALLATION AND COMPILATION OF THE K-BUS DRIVER.....	6
STEP 2 : EXAMPLE – CASE OF APPLICATION.....	8
AUTHOR'S NOTE.....	20

GENERAL INFORMATION

1. **Description:** This "How to" shows the installation and compilation of the K-Bus driver in the CX1020 unit (Beckhoff) with Linux (Xubuntu 8.04) / RTAI (3.8). The driver has been developed under Comedi format. This how to includes an application example that describes the code generation from a PC (Host) with Matlab®/Simulink®, and the compilation and execution in the CX1020 unit.

2. **Sources / References:**

RTAI 3.4 User Manual rev 0.3

<https://www.rtai.org> ; Documentation / Reference Documents

Comedi – Template for writing drivers (skel.c)

David Schleef

www.comedi.org

Beckhoff Driver (CX-1100)

Richard Lemon

<http://sourceforge.net/projects/beckhoffdriver/>

Data sheets Beckhoff devices: CX1020, CX1100-002, KL1408, KL2408, KL3404 and KL4034.

<http://www.beckhoff.com>

IMPORTANT:

USE THIS INFORMATION UNDER YOUR OWN RISK. THE GICI IS NOT RESPONSIBLE FOR ANY DAMAGE OR DEGRADATION IN THE EQUIPMENTS OR DEVICES, NEITHER FOR DAMAGE THAT THEY CAN PRODUCE A THIRD PARTY.

PRELIMINARIES

1. Hardware:

CX-1020-0121	→	CPU (Intel® Celeron® M ULV 1 Ghz) 2 RJ45 Sockets 2 USB ports + 1 DVI port 1GB RAM (Modified, factory settings 256 MB).
CX-1100-0002	→	Power supply for CPU and terminals E/S K-Bus.
KL3404	→	Analog Inputs Terminal, 4 channels, +/- 10 V.
KL4034	→	Analog Outputs Terminal, 4 channels, +/- 10 V.
Compact Flash	→	8.0 GB.

2. Connection accessories and complementary equipments:

Power Supply 24V (output)

USB-CD reader

USB (Hub)

Keyboard and Mouse, both USB

Display

3. Software:

3.1. Operative System → Linux (Xubuntu 8.04) / RTAI (RTAI 3.8 con RTAI-Lab).

3.2. Attached files:

kbusdrv.c	→	Driver K-Bus
loadrtaimodules.sh	→	Script to load kernel modules RTAI– Comedi
unloadrtaimodules.sh	→	Script to unload kernel modules RTAI – Comedi

Availables in: <http://gici.drupalgardens.com/content/descargas>

4. Others:

4.1. It is highly recommendable to realize the RTAI installation in CX1020 unit as shown in document “How to install RTAI in Beckhoff CX1020”. Available in:

<http://gici.drupalgardens.com/content/descargas>

4.2. Knowledge about K-Bus operation. See data sheet CX1100-002 power supply, available in::

www.beckhoff.com

STEP 1: INSTALLATION AND COMPILATION OF THE K-BUS DRIVER

Notes: The K-Bus driver has been developed under Comedi interface with the goal to access to I/O signals of the terminals (KL1408, KL2408, KL3404, KL4034). However, it's possible to modify it for other terminals.

For more information about the driver development, contact to: <http://gici.drupalgardens.com/contact>

The values selected in the setup option below are chosen by the author. However according to your preferences and own experience, these values can be changed.

1. Start the system CX1020.
2. Copy the file supplied "kbusdrv.c" into the directory " /usr/src/comedi/comedi/drivers ".
3. Open a terminal and login as super-user.
4. Go to the directory " /usr/src/comedi/comedi/drivers "

```
# cd /usr/src/comedi/comedi/drivers
```

5. Edit the file "Makefile.am"

- 5.1. Open the file with the preferred text editor (in this case "Nano")

```
# nano Makefile.am
```

- 5.2. Make the following changes:

- a) In the section "module_PROGRAMS =" after the line:

```
jr3_pic.ko \
```

```
insert:
```

```
kbusdrv.ko \
```

- b) In the section "SOURCES" after the line:

```
jr3_pci_ko_SOURCES = jr3_pci.c
```

```
insert:
```

```
kbusdrv_ko_SOURCES = kbusdrv.c
```

- 5.3. Save the changes and exit (for "Nano")

```
Ctrl + x
```

```
"Y"
```

```
"Enter"
```

6. Edit the file “Kbuild” in the same directory (/usr/src/comedi/comedi/drivers):

- 6.1. Open the file

```
# nano Kbuild
```

- 6.2. Make this change:

After the line:

```
obj-m += jr3_pic.o
```

Insert:

```
obj-m += kbusdrv.o
```

- 6.3. Save the changes and exit

```
Ctrl + x
```

```
“Y”
```

```
“Enter”
```

7. Go to the directory “ /usr/src/comedi “

```
# cd /usr/src/comedi
```

8. Edit the file Makefile.am

- 8.1. Open the file:

```
# nano Makefile.am
```

- 8.2. Make the change:

In the line:

```
COMEDI_DRIVER_FILES := $(shell find $(srcdir)/comedi/drivers -name '*.c' | grep -v  
mod\.c | sort )
```

Modify:

```
COMEDI_DRIVER_FILES = /usr/src/comedi/comedi/drivers -name '*.c' | grep -v  
mod\.c | sort )
```

- 8.3. Save the change and exit:

```
Ctrl + x
```

```
“Y”
```

```
“Enter”
```

9. Execute:

```
# ./autogen.sh
```

```
# make
```

At this point, the K-Bus driver has been compiled and it can be accessed via Comedi. For more information about how linking the driver with applications, visit: www.comedi.org

STEP 2 : EXAMPLE – CASE OF APPLICATION

The following example presents how to generate code from a PC (host) for compilation and execution on the CX1020 unit (Target).

The first part shows the making of the model in Simulink® and the code generation from Matlab® in the PC (Host). The second one corresponds to the compilation and execution code for RTAI in the CX1020 unit (Target).

Specification of the System

The response to a step input for continuous system with unity feedback is going to be calculated. The plant transfer function $G_c(s)$ is:

$$G_c(s) = \frac{25}{0.7 s^2 + 3.5 s}$$

1. Discretization of the system, building the model in Simulink® and code generation.

1.1. Open Matlab® in the PC (Host)

1.2. On the command window discretize $G_c(s)$ with a sampling time $T_s = 2$ ms.

```
>> Gc = tf ( [ 25 ] , [ 0.7 3.5 0 ] )
```

Transfer function:

$$\frac{25}{0.7 s^2 + 3.5 s}$$

```
>> Ts = 0.002
```

```
>> Gd = c2d( Gc, Ts, 'zoh')
```

Transfer function:

$$\frac{7.119e-005 z + 7.095e-005}{z^2 - 1.99 z + 0.99}$$

Sampling time (seconds) : 0.002

1.3. Find the system response parameters to a unitary step input (close loop with unitary feedback) with the command “stepinfo()”.

```
>> stepinfo ( feedback ( Gd , 1 ) )
```

ans =

RiseTime:	0.2480	[s]
SettlingTime:	1.4060	[s]
SettlingMin:	0.9013	[un]
SettlingMax:	1.2383	[un]
Overshoot:	23.8260	[%]
Undershoot:	0	
Peak:	1.2383	[un]
PeakTime:	0.5780	[s]

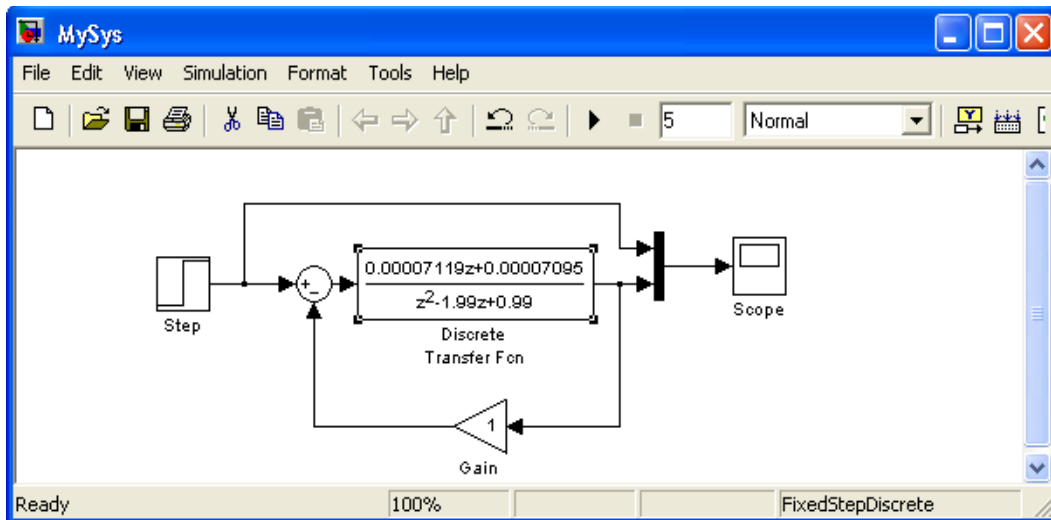
1.4. Create the model in Simulink® with Gd coefficients :

a) Create the directory “Demo” into “ C:\...\My Documents\MATLAB\ ”

b) Open Simulink®, create a new model and save as “MySys.mdl”, in :

“C:\...\My Documents\MATLAB\Demo”

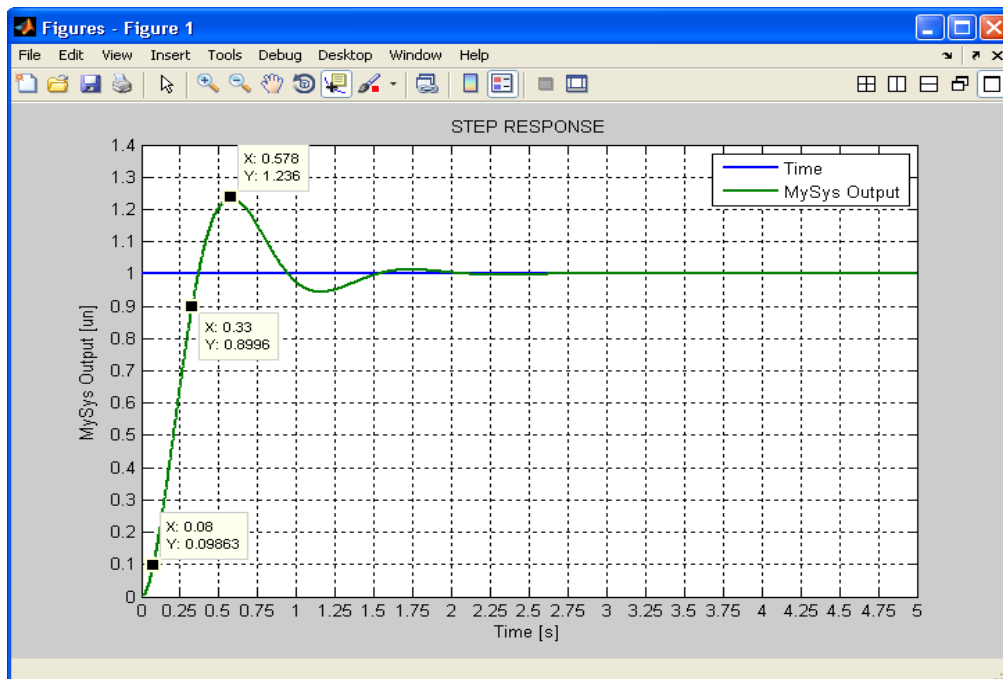
c) Build the below diagram:



d) Adjust the following parameters on the block:

Discrete Transfer Fcn → Numerator coefficients: [7.119e-5 7.095e-5]
 Denominator coefficients: [1 -1.99 0.99]
 Sample time: 0.002

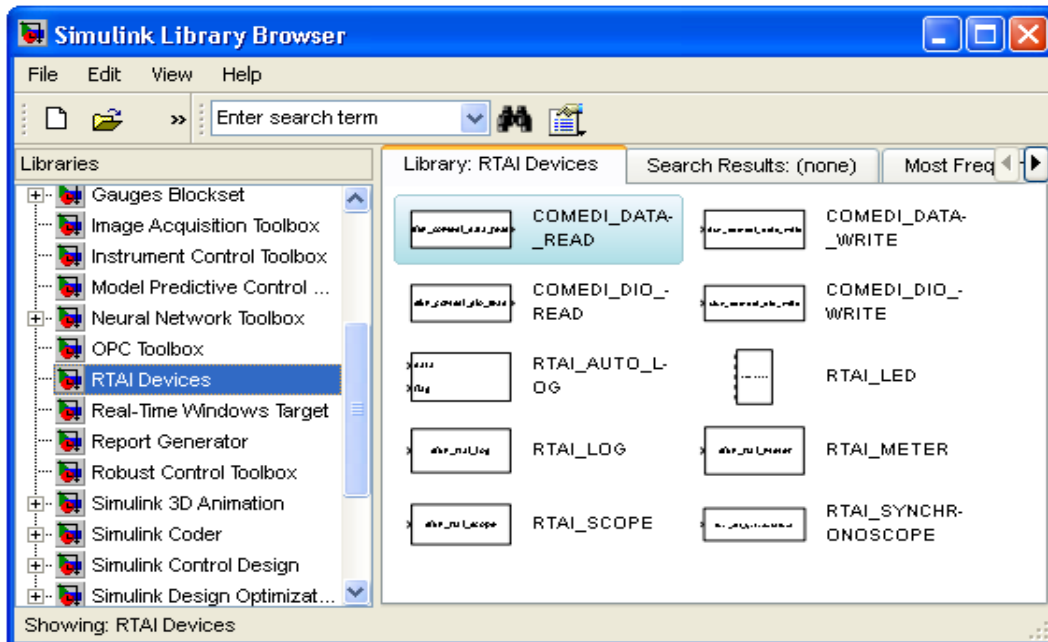
e) For a 5 second simulation time, the following result should be displayed.



It can be seen that the trace values correspond to the values obtained in the item 1.3. (Peak, Peak Time and Rise Time (0,33-0,08 $\approx$ 0,25)).

Once the system is verified, insert the RTAI blocks to link both the "I/O model" with the physical I/O of the target, and the RTAI-Lab instrumentation with the target.

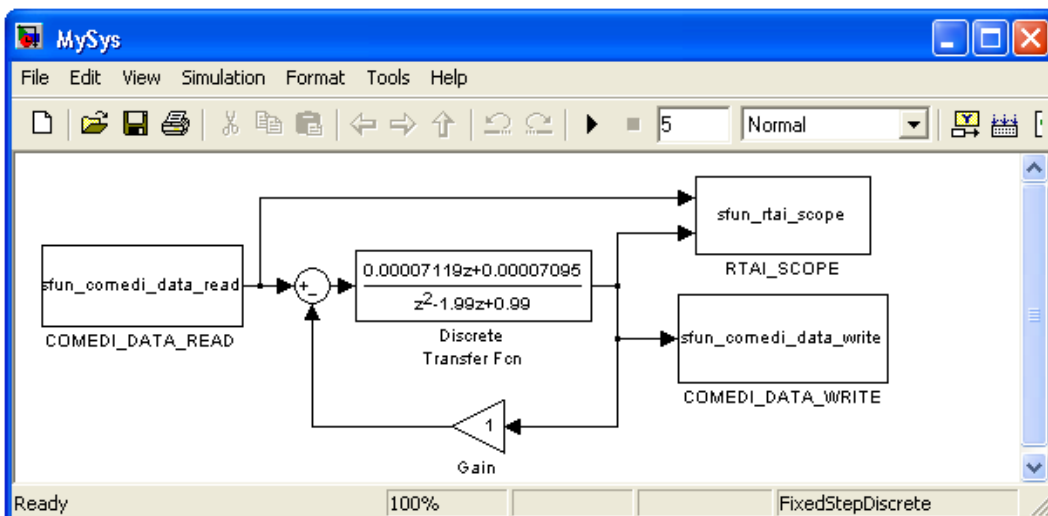
- f) Insert RTAI blocks from the "RTAI Devices" library in the Simulink® "Library Browser".



The COMEDI_* blocks are used to read/write the I/O signals both the analog and the digital ones. The RTAI_* blocks are used to implement visualization and log devices for RTAI-Lab.

This example uses: COMEDI_DATA_READ (to read the input values of the KL3404 terminal), COMEDI_DATA_WRITE (to write the output values in the KL4034 terminal), and RTAI_SCOPE (to visualize the system signals in RTAI-Lab).

Modify "MySys" adding the aforementioned blocks:



g) Parametrize the inserted blocks with the following values:

COMEDI_DATA_READ →

Device: comedi0

Channel: 2 (It means that, It's going to use the input 3 of the KL3404 terminal. In Comedi, the channel number starts in zero).

Range: 0 (The KL3404 terminal has only one voltage range +/-10V. Leave this value (0) because only one range has been defined in the driver)

Analog reference: ground

Sampling time: 0.002 (Sample Time of the system [Ts]).

COMEDI_DATA_WRITE →

Device: comedi0

Channel: 1 (It means that, It's going to use the output 2 of the KL4034 terminal. In Comedi, the channel number starts in zero).

Range: 0 (The KL4034 terminal has only one voltage range +/-10V. Leave this value (0) because only one range has been defined in the driver)

Analog reference: ground

RTAI_SCOPE →

Number of traces: 2 (One signal by channel, one for the input and one for the output).

h) Save the changes.

1.5. Code Generation.

In “ Simulation / Configuration Parameters ” of “ MySys “, set the following values:

SOLVER:

Solver options →

Type: Fixed-step

Solver: Discrete (no continuous states)

Fixed-step size (fundamental sample time): 0.002 ← [Ts]

CODE GENERATION:

Target Selection →

System target file: rtai.tlc

Language: C

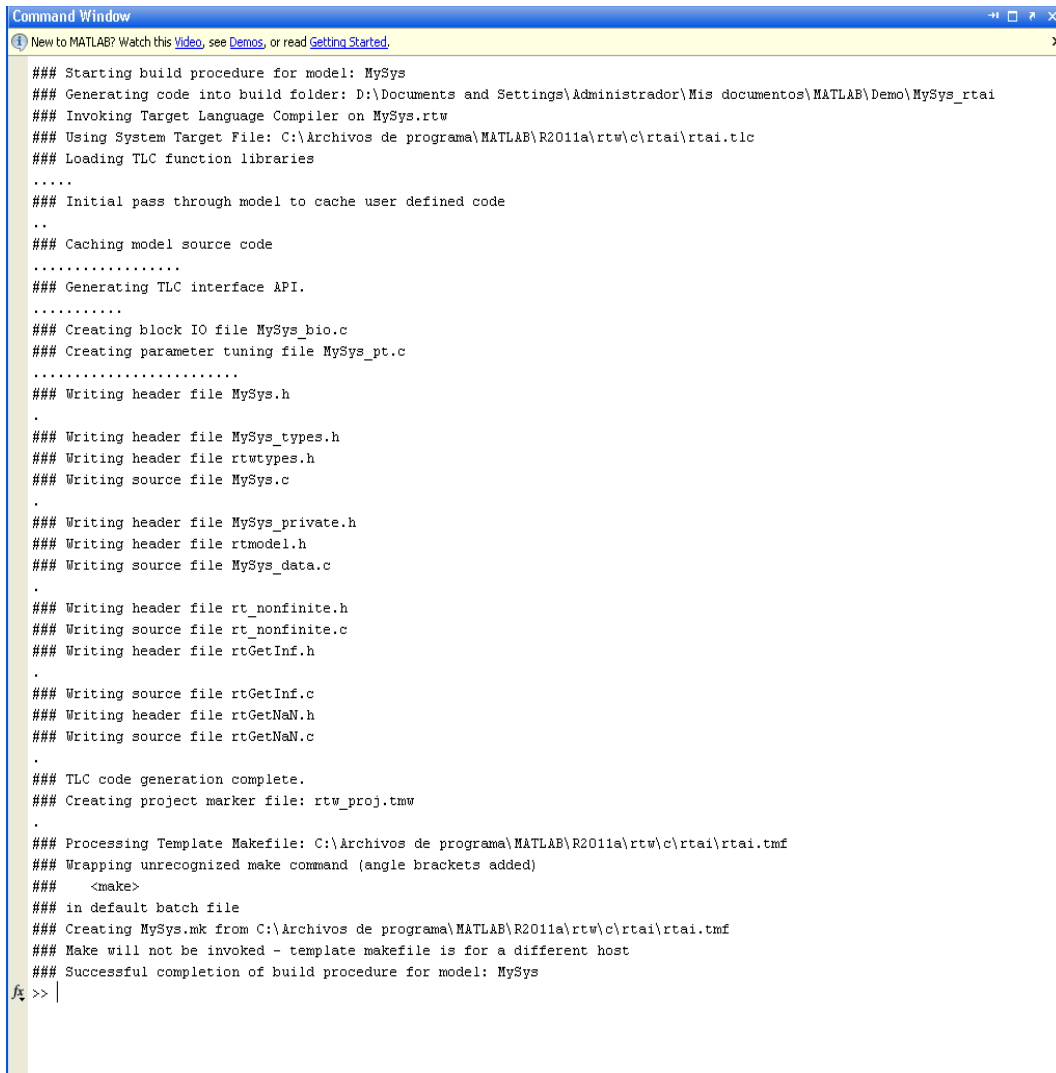
Description: Linux RTAI Real-Time Target

Makefile configuration →

Generate makefile: [OK]
 Make command: make_rtw
 Template makefile: rtai.tmf

Finally: Click on “Build” or Ctrl + b

Matlab command window shows the process status.



```

Command Window
New to MATLAB? Watch this Video, see Demos, or read Getting Started.

### Starting build procedure for model: MySys
### Generating code into build folder: D:\Documents and Settings\Administrador\Mis documentos\MATLAB\Demo\MySys_rtai
### Invoking Target Language Compiler on MySys.rtw
### Using System Target File: C:\Archivos de programa\MATLAB\R2011a\rtw\c\rtai\rtai.tlc
### Loading TLC function libraries
.....
### Initial pass through model to cache user defined code
..
### Caching model source code
.....
### Generating TLC interface API.
.....
### Creating block IO file MySys_bio.c
### Creating parameter tuning file MySys_pt.c
.....
### Writing header file MySys.h
.
### Writing header file MySys_types.h
### Writing header file rtwtypes.h
### Writing source file MySys.c
.
### Writing header file MySys_private.h
### Writing header file rtmodel.h
### Writing source file MySys_data.c
.
### Writing header file rt_nonfinite.h
### Writing source file rt_nonfinite.c
### Writing header file rtGetInf.h
.
### Writing source file rtGetInf.c
### Writing header file rtGetNaN.h
### Writing source file rtGetNaN.c
.
### TLC code generation complete.
### Creating project marker file: rtw_proj.tmw
.
### Processing Template Makefile: C:\Archivos de programa\MATLAB\R2011a\rtw\c\rtai\rtai.tmf
### Wrapping unrecognized make command (angle brackets added)
### <make>
### in default batch file
### Creating MySys.mk from C:\Archivos de programa\MATLAB\R2011a\rtw\c\rtai\rtai.tmf
### Make will not be invoked - template makefile is for a different host
### Successful completion of build procedure for model: MySys

fx >>
  
```

1.6. The preceding procedure creates the directories “MySys_rtai” and “slprj”, in:

“ C:\...\My Documents\MATLAB\Demo ”.

At this point, the code generation of the model ends.

2. Compilation and execution of the model in the CX1020 unit (target):

2.1. In the PC (Host) copy the directory “MySys_rtai” into a PenDrive. Directory placed in:

" C:\...\Mis Documentos\MATLAB\Demo ”.

2.2. Remove the PenDrive from the PC (Host). Turn on the CX1020 unit (Target), login into the system, and plug the PenDrive.

2.3. The internal structure of the K-Bus driver has a real time task to perform the read/write cycle. The use of a real time task implies starting a real time timer (rt_start_timer()).

Then, any application that uses the K-Bus driver must ask before starts if there is a "hard real time" timer running. Remember, if the timer is started more then once the second call resets the timer and starts with the new period. (See RTAI 3.4 User Manual rev 0.3. -pag 36).

This is the reason why it is necessary to modify the file "rtmain.c" placed on (/usr/local/matlab/rw/c/rtai).

a) Open a terminal, login as super-user and open the file:

```
# nano /usr/local/matlab/rw/c/rtai/rtmain.c
```

b) Go to the “else” sentence in the line 889 (corresponding to “if (!ClockTick)” in line 885), and make the modification shown below:

Original Code:

```
    } else {
        rt_set_periodic_mode();
        rt_BaseRateTick = start_rt_timer ( nano2count( rt_BaseTaskPeriod ) );
    }
```

Modification:

```
    } else {
        if (rt_is_hard_timer_running() == 1)
        {
            printf("\nrt_timer already running\n");
            rt_BaseRateTick = nano2count(rt_BaseTaskPeriod);
        }
        else
        {
            // printf("\nNo rt_timer running\n");
            rt_set_periodic_mode();
            rt_BaseRateTick = start_rt_timer(nano2count(rt_BaseTaskPeriod));
        }
    }
```

c) Save the changes and exit

Ctrl + x

“Y”

“Enter”

2.4. Copy the following files in the directory "models" placed in "/usr/local/src/".

Note: If the "models" directory doesn't exist, create it.

- MySys_rtai (PenDrive)
- loadrtaimodules.sh
- unloadrtaimodules.sh

2.5. The "loadrtaimodules.sh" and "unloadrtaimodules.sh" scripts perform the kernel modules load/unload to execute RTAI-Lab and Comedi. The last line of "loadrtaimodules.sh" performs the link between K-Bus driver and Comedi using "/dev/comedi0" (as specified in the item 1.4.g. "Device: comedi0").

2.6. Go to the directory "MySys_rtai" placed in "/usr/local/src/models "

```
# cd /usr/local/src/models/MySys_rtai
```

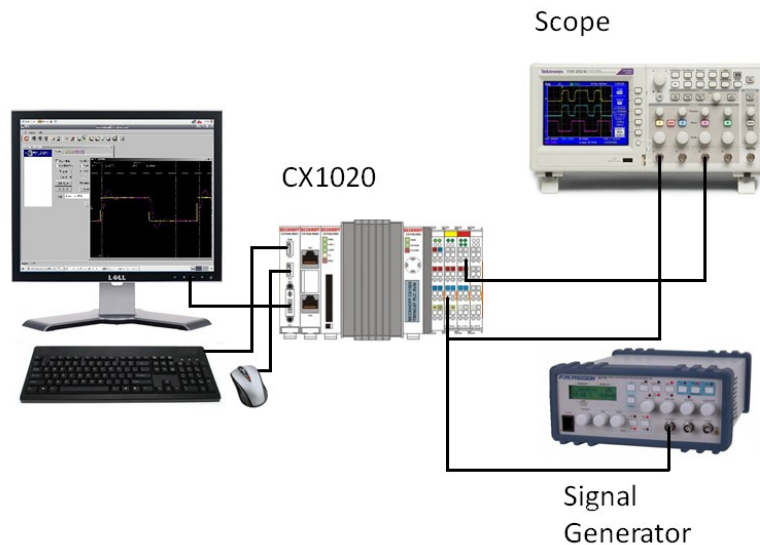
2.7. Execute

```
# make -f MySys.mk
```

2.8. Go to the directory "/usr/local/src/models "

```
# cd /usr/local/src/models
```

2.9. Plug the equipments as shown below:



- a) Signal Generator: Set a pulse train of frequency 0.125 Hz (period 8 s), 50% duty cycle and 1V amplitude.
- b) Wire the Signal Generator and the Scope N°1 channel to the KL3404 terminal N°3 input channel.
- c) Wire the KL4034 output Channel N°2 to the Scope channel N°2.

Now the model can run for both Direct Mode or RTAI-Lab.

2.10. Execution of the model – Direct Mode (without graphical interface).

- a) From the directory “ /usr/local/src/models “, load the modules for RTAI and Comedi:

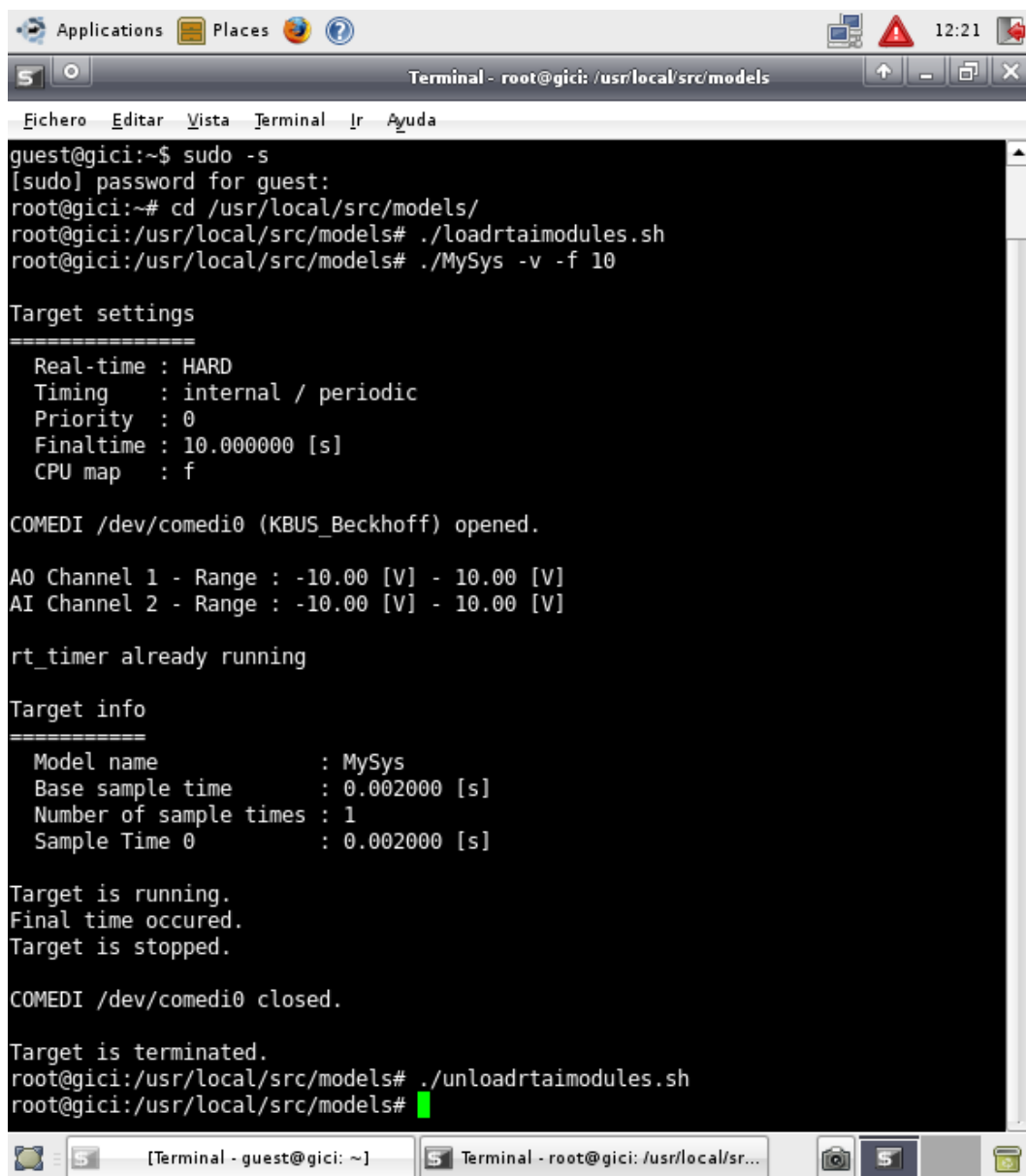
```
# ./loadrtaimodules.sh
```

- b) Execute the model (for 10 seconds)

```
# ./MySys -v -f 10
```

Note: If you want to execute in continuous way, try this `# ./MySys -v -f`

- c) The process report is shown below:



```
guest@gici:~$ sudo -s
[sudo] password for guest:
root@gici:~# cd /usr/local/src/models/
root@gici:/usr/local/src/models# ./loadrtaimodules.sh
root@gici:/usr/local/src/models# ./MySys -v -f 10

Target settings
=====
Real-time : HARD
Timing    : internal / periodic
Priority   : 0
Finaltime : 10.000000 [s]
CPU map   : f

COMEDI /dev/comedi0 (KBUS_Beckhoff) opened.

AO Channel 1 - Range : -10.00 [V] - 10.00 [V]
AI Channel 2 - Range : -10.00 [V] - 10.00 [V]

rt_timer already running

Target info
=====
Model name       : MySys
Base sample time : 0.002000 [s]
Number of sample times : 1
Sample Time 0    : 0.002000 [s]

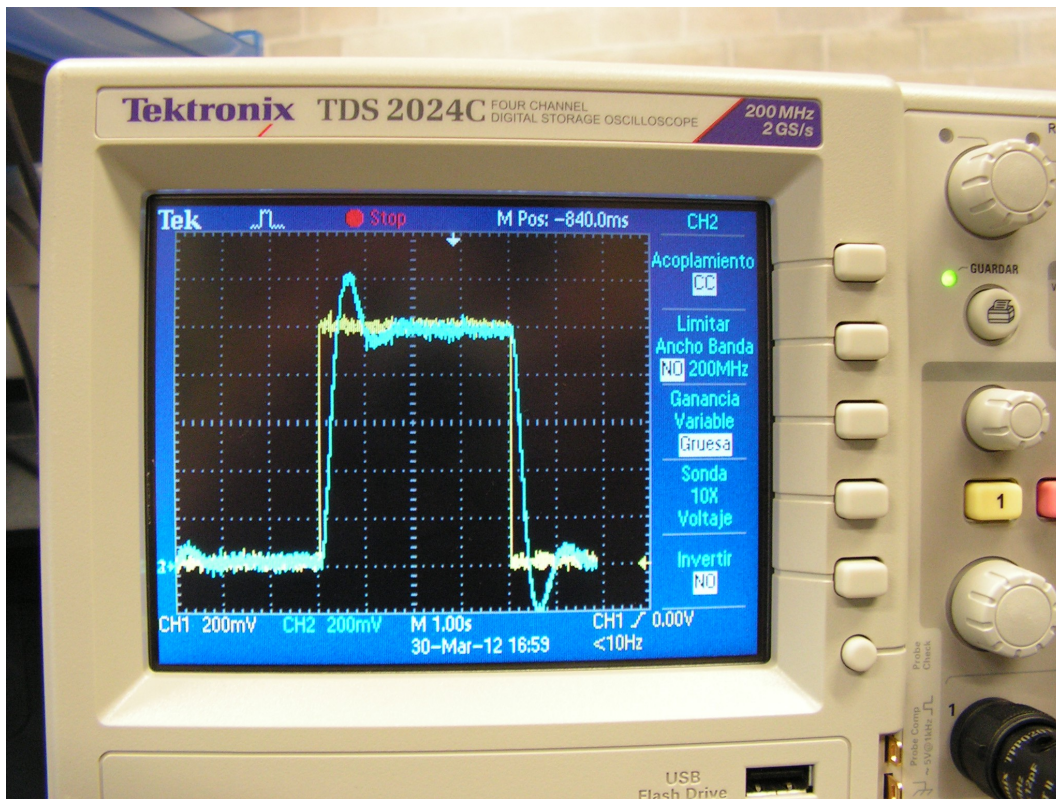
Target is running.
Final time occurred.
Target is stopped.

COMEDI /dev/comedi0 closed.

Target is terminated.
root@gici:/usr/local/src/models# ./unloadrtaimodules.sh
root@gici:/usr/local/src/models#
```

The report shows the Target Settings, information related to the K-Bus driver (status, and signals specifications (type / range / value)), model information (name, sample time) and finally the state of the process.

d) Signals taken from the KL3404 and the KL4034 terminals.



The model behavior corresponds to the model simulated in the PC (Host)

The channel N.1 signal (yellow) is the system input (train pulse with 1V amplitude, period of 8s and 50% duty cycle). The channel N.2 signal (cyan) is the system output, whose estimate parameters are:

$$\text{Peak} \approx 6 [\text{un}] * 200 [\text{mV}]/[\text{un}] \approx 1.2 \text{ V}$$

$$\text{Peak Time} \approx 0.6 [\text{un}] * 1.00 [\text{s}]/[\text{un}] \approx 0.6 \text{ s}$$

The estimated values are consistent with those obtained in simulation (item 1.3.).

e) To see the status of the running real time tasks:

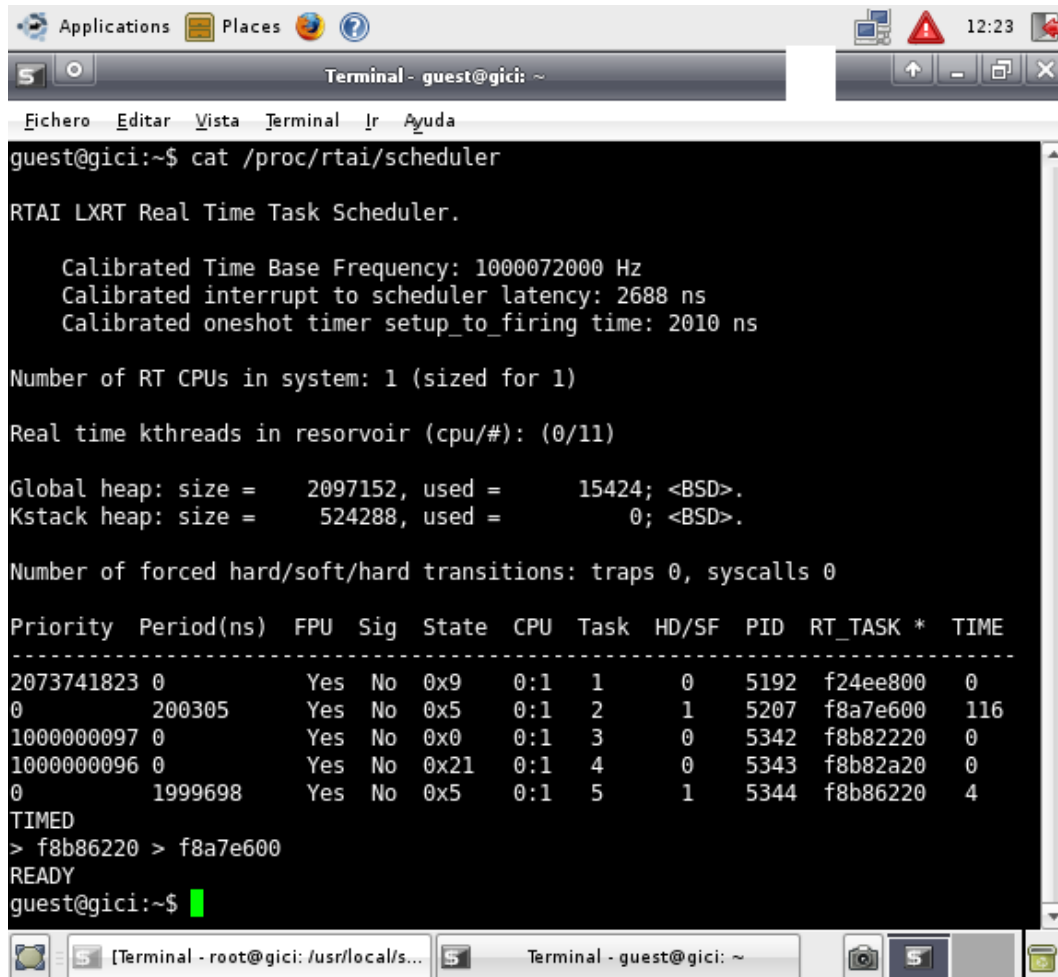
- Execute the model again (repeat the item 2.10.b) for 60 seconds

```
# ./ MySys -v -f 60
```

- Once the model is running, open a new terminal and run:

```
$ cat /proc/rtai/scheduler
```


This command visualizes the parameters of: RTAI scheduler calibration and the real time tasks (priority, period, status, id, etc).



```

guest@gici:~$ cat /proc/rtai/scheduler

RTAI LXRT Real Time Task Scheduler.

  Calibrated Time Base Frequency: 1000072000 Hz
  Calibrated interrupt to scheduler latency: 2688 ns
  Calibrated oneshot timer setup_to_firing time: 2010 ns

Number of RT CPUs in system: 1 (sized for 1)

Real time kthreads in reservoir (cpu/#): (0/11)

Global heap: size = 2097152, used = 15424; <BSD>.
Kstack heap: size = 524288, used = 0; <BSD>.

Number of forced hard/soft/hard transitions: traps 0, syscalls 0

Priority  Period(ns)  FPU  Sig  State  CPU  Task  HD/SF  PID  RT_TASK *  TIME
-----
2073741823 0          Yes  No  0x9   0:1  1     0     5192 f24ee800  0
0          200305    Yes  No  0x5   0:1  2     1     5207 f8a7e600 116
1000000097 0          Yes  No  0x0   0:1  3     0     5342 f8b82220  0
1000000096 0          Yes  No  0x21  0:1  4     0     5343 f8b82a20  0
0          1999698    Yes  No  0x5   0:1  5     1     5344 f8b86220  4
TIMED
> f8b86220 > f8a7e600
READY
guest@gici:~$

```

For example, the tasks with PID 5207 and 5344 correspond to the K-Bus driver and MySys model. Those tasks have the same priority (0 the highest) and periods of 200 us (K-Bus) and 1.99 ms (MySys) respectively.

Note: For more information about this parameters, see RTAI 3.4 User Manual rev 0.3.

f) Finally, download the RTAI and Comedi kernel modules:

```
# ./unloadrtaimodules.sh
```

2.11. Execution of the model – RTAI-Lab.

The special feature of this method is the graphical interface.

In the directory “usr/local/src/models”

a) Load the RTAI and Comedi kernel modules.

```
# ./loadrtaimodules.sh
```

b) Start the model in “wait” mode:

```
# ./MySys &
```

Appears on the screen:

```
COMEDI /dev/comedi0 (KBUS_Beckhoff) opened.
```

```
AO    Channel 1 – Range : -10.00 [V] – 10.00 [V]
```

```
AI    Channel 2 – Range : -10.00 [V] – 10.00 [V]
```

```
rt timer already running
```

```
[1] 5271
```

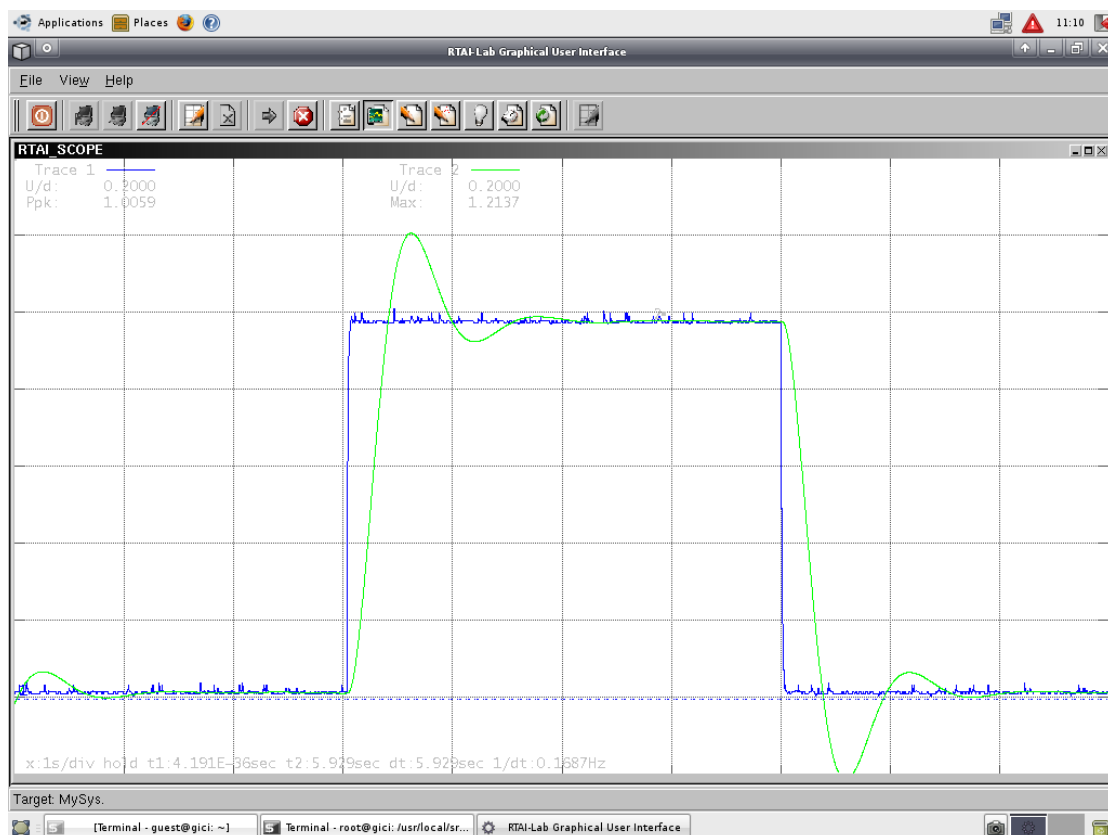
The K-Bus driver has been opened and the model "MySys" uses the channel N°1 analog output (channel N°2 of the KL4034 terminal) and the Channel N°2 analog inputs (channel N°3 of the KL3404 terminal). "MySys" model PID is 5271.

c) Start RTAI-Lab:

```
# /usr/realtime/bin/xrtailab &
```

Once RTAI-Lab interface is opened, connect to the Target (File -> Connect... + OK) and enable the Scope (View -> Scopes [OK]). Set the time axis with 1 Sec/Div and the amplitude with 1 V/Div for both channel 1 and 2.

The system response is similar to the PC (host) model response.



- d) To finish, stop the "Scope", then disconnect the Target "File → Disconnect" and finally "File → Quit"
- e) Download the RTAI and Comedi kernel modules.

./unloadrtaimodules.sh

AUTHOR'S NOTE

For any questions or comments contact:

- <http://gici.drupalgardens.com/contact>
- j.andres.cubillos@gmail.com

Any suggestion to improve this document will be appreciated.